

Final Project Report

Project Proposal And Requirements Analysis

1. Team Information

Name	Student number	Main responsibilities
Lennart Axel Conrad	2025080264	Chess-domain architecture, C++ core implementation, neural-network/evaluator paths, tests, model validation, build/package work, documentation integration.
Erik Mkrtchyan	2025080273	raylib interaction workflows, visual layout review, activation presentation, manual UI testing, tutorial/demo preparation, user-facing documentation review.

Both team members should be able to explain the build process, major classes, source layout, runtime workflow, tests, and final demo behavior.

2. Project Topic

The project is a native C++ chess neural-network visualizer. It combines a complete legal human-vs-human chess board with visualization panels for:

- NNUE HalfKP accumulator evaluation,
- LC0-style CNN residual evaluation,
- LC0 BT4-style token-transformer evaluation,
- handcrafted Classical evaluation.

It also includes a native PUCT MCTS search-tree workbench. The topic is more advanced than a basic chess game because it includes legal move generation, notation, file I/O, binary model loading, tensor operations, search, tests, and real-time visualization.

3. Prior Work Used As Reference

The submitted runtime is C++. Prior personal chess projects were used as reference material only:

- <https://github.com/LenniAConrad/chess-rtk>: Java chess research toolkit with move generation, notation, evaluation, engine/search, datasets, and visualization concepts.
- <https://github.com/LenniAConrad/chess-web>: TypeScript chess player/trainer with browser UI and gameplay workflows.

The C++ application does not call either repository at runtime.

4. Course Requirement Mapping

Requirement	Project evidence
Classes and encapsulation	<code>Position</code> , <code>Game</code> , <code>MoveHistory</code> , <code>Tensor</code> , <code>ActivationSnapshot</code> , <code>Nnue/Cnn/Bt4 Network</code> , <code>Mcts</code> , <code>App</code> , <code>BoardView</code> , activation views, editor widgets.
Multi-file C++ structure	Separate <code>.h/.cpp</code> files under <code>src/chess</code> , <code>src/game</code> , <code>src/io</code> , <code>src/nn</code> , <code>src/search</code> , and <code>src/viz</code> .
File I/O	<code>Config</code> load/save, FEN load/save, PGN export helpers, binary model readers, test reference data, Windows package config/assets/models.
Inheritance/polymorphism	<code>INetwork</code> implemented by <code>NNUE/CNN/BT4</code> networks; <code>IActivationView</code> implemented by <code>NNUE/CNN/BT4/Classical</code> views.
Templates	<code>Tensor<T, Rank></code> generic tensor container.
Linked list	<code>MoveHistory</code> manually manages a doubly linked list of move records for undo/redo.
Arrays/containers	Bitboards, fixed move arrays, mailbox board, STL vectors/maps/unordered maps for histories, tensors, snapshots, model weights, and search nodes.
Chess game	Complete human-vs-human legal board with promotion, undo/redo, move history, FEN, and setup editor.
Advanced feature	NN/evaluator activation visualizer plus native PUCT MCTS tree workbench.
User manual	<code>docs/user-manual.md</code> .
Design specification	<code>docs/design-spec.md</code> and <code>docs/class-diagram.png</code> .
Test cases	<code>docs/test-cases.md</code> , <code>tests/*.cpp</code> , and screenshots under <code>docs/screenshots/</code> .
Summary report	<code>docs/summary-report.md</code> .
AI usage	<code>docs/ai-usage.md</code> .
Demo media	<code>docs/tutorial.gif</code> , <code>docs/demo.mp4</code> , and screenshot stills under <code>docs/screenshots/</code> .
Packaged build	Linux build scripts and Windows x64 package under <code>dist/</code> .

5. Functional Requirements

1. The user can play a legal chess game with mouse input.
2. The board shows selected pieces, legal destinations, last move highlights, promotion choices, and status text.
3. The user can undo, redo, reset, flip, and play a random legal move.
4. The user can load a complete FEN string and save the current FEN.
5. The user can create custom legal positions in setup mode and cannot apply invalid positions.
6. The user can switch between NNUE, CNN, BT4, Classical, and Off activation modes without changing the chess position.
7. The visualizer displays meaningful architecture-specific data for each evaluation path.
8. The user can run and inspect a live search preview from the current position.
9. The user can open the MCTS tree workbench, run PUCT search, pan/zoom the tree, filter it, scrub growth frames, and trace selected nodes.
10. The project can be built, tested, documented, and packaged from command line scripts.

6. Non-Functional Requirements

- The chess core must be independent of raylib.
- CPU execution must be available without CUDA.
- Runtime code must not depend on Java, TypeScript, Python, external chess engines, or online services.
- Compact runtime model files should keep the demo portable.
- The UI should remain responsive while model evaluation and search run.
- The documentation should match the current source, config, and model files.
- The test suite should be runnable from a clean command line build.

7. Acceptance Plan

Before submission or defense:

1. Run `./scripts/test.sh`.
2. Run `./build/tests/cnnv_tests` if an explicit individual-test count is needed.
3. Run `./scripts/demo_check.sh`.
4. Launch the app with `./scripts/run.sh`.
5. Demonstrate board play, undo/redo, FEN load/save, setup validation, architecture switching, search preview, and MCTS tree view.
6. Regenerate PDFs with `./scripts/export_docs.sh` when Markdown documents are changed.
7. Rebuild the Windows package when source, assets, config, or model defaults change.

Design Specification

1. Purpose

`cpp-nn-visualizer` is a C++17 desktop application for legal chess play and model introspection. The program has two goals:

1. Provide a complete human-vs-human chess board for Tutorial III and the final project demo.
2. Visualize how several chess evaluation families process the current position: NNUE, LC0 CNN, LC0 BT4, and a handcrafted Classical evaluator.

The program is intentionally split so chess rules and model inference can be tested without raylib, while the raylib layer only handles presentation and input.

2. Build Targets

The executable is assembled from two internal static libraries:

Target	Responsibility
<code>cnnv_core</code>	Chess rules, game state, file I/O, tensor operations, NN loaders, NN forward passes, Classical evaluator, and MCTS.
<code>cnnv_viz</code>	raylib UI, board rendering, controls, dialogs, activation views, editor, and tree visualization.
<code>cnnv</code>	Thin executable entry point linking <code>cnnv_viz</code> and <code>cnnv_core</code> .
<code>cnnv_tests</code>	Headless test binary run through CTest.

This split prevents raylib from leaking into the chess and NN modules.

3. Class Diagram

The high-level class diagram is generated from `docs/class-diagram.dot`.

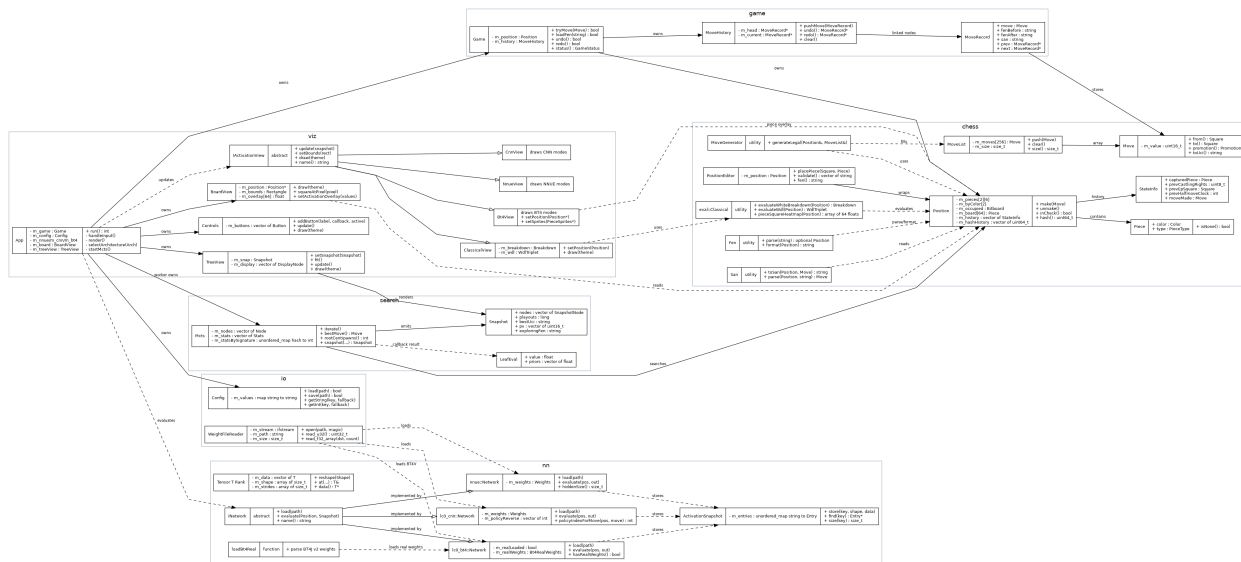


Figure 1: Core class diagram

4. Module Responsibilities

4.1 src/chess

The chess module owns all legal chess rules:

- **Position**: board state, side to move, castling, en-passant, clocks, make and unmake, check/draw helpers, and hash history.
- **Move**: packed UCI-style move representation.
- **MoveList**: fixed-capacity legal move container.
- **MoveGenerator**: legal move generation with check filtering.
- **Fen** and **San**: notation parsing/formatting.
- **PositionEditor**: mutable editor facade with validation.
- **Perft** and **Zobrist**: rule regression and hashing helpers.

The implementation uses bitboards for fast attacks and occupancy, plus a 64-square mailbox array for direct UI/parser lookup.

4.2 src/chess/eval

The Classical evaluator is a model-free C++ port of the chess-rtk handcrafted evaluator. It returns:

- an 11-term white-perspective centipawn breakdown,
- a side-to-move WDL triplet,
- per-piece piece-square tables,
- an occupied-square PST heatmap for the board overlay.

The visualizer uses this as its own activation choice and as a fallback evaluation path for search.

4.3 src/game

The game module wraps **Position** into a user-facing game:

- **Game**: legal move application, undo/redo, FEN loading, status, move list jumps, and game reset.
- **MoveHistory**: a manually implemented doubly linked list of move records.
- **Pgn**: PGN export helpers.

All normal board mutations go through **Game**, which keeps SAN, FEN snapshots, and undo/redo state consistent.

4.4 `src/io`

The I/O module contains:

- **Config**: flat key/value config loader and saver.
- **FenIo**: FEN file save/load helpers.
- **WeightFileReader**: little-endian binary reader for model blobs.

Runtime configuration is read once at startup from `config.ini`.

4.5 `src/nn`

The shared NN layer contains:

- **Tensor<T, Rank>**: rank-aware template tensor container.
- **ActivationSnapshot**: string-keyed activation tensor store.
- **INetwork**: polymorphic evaluator interface.
- **ops/**: CPU tensor operations such as convolution, matmul, attention, batch norm, layer norm, softmax, and activation functions.
- **CudaOps**: optional CUDA acceleration hooks; CPU remains the required path.

Each network writes named tensors into an **ActivationSnapshot**. The UI copies only the tensors it needs.

4.6 `src/nn/nnue`

The NNUE module implements:

- HalfKP feature encoding,
- accumulator refresh,
- compact NNUE binary loading,
- clipped activation and output contribution calculation,
- learned-weight atlas tensors for the NNUE panel.

The default file is `models/nnue-halfkp-demo.bin`.

4.7 `src/nn/lc0_cnn`

The CNN module implements:

- canonical LC0 112-plane input encoding,
- LC0J binary loading,
- residual trunk forward pass,
- policy head with compressed 4672-entry policy map,
- WDL/value head,
- final board heatmap and per-layer tensor snapshots.

The default file is `models/lc0-cnn-small-112p-4x32-policy4672-wdl3.bin`. It has 4 residual blocks, 32 trunk channels, 73 policy planes, 4672 policy logits, and 3 WDL outputs.

4.8 `src/nn/lc0_bt4`

The BT4 module has two paths:

Path	Use
BT4J v2 real path	Loads compact CRTK BT4 weights and runs a native transformer forward pass.
BT4V synthetic fallback	Validates legacy metadata and generates deterministic transformer-like activations if no real BT4J model is loaded.

The default file is `models/lc0-bt4-tiny-96x4x4h.bin`. It is a compact real BT4J v2 model generated by `scripts/generate_small_bt4.py` with:

- 112 canonical input channels,
- 64 board tokens,
- 96-dimensional token embeddings,
- 4 encoder blocks,
- 4 attention heads,
- attention policy head to 1858 logits,
- WDL value head.

The loader and forward structs are written to support more BT4J components than the tiny default uses, including optional smolgen records and larger topologies. The official LC0 protobuf file format is not parsed at runtime.

4.9 src/search

`Mcts` is a single-threaded PUCT search engine used by the tree workbench. It is independent of raylib and accepts an evaluator callback:

```
using Evaluator =
    std::function<LeafEval(const Position&, const MoveList&)>;
```

The app supplies a callback that evaluates leaves through the selected architecture where possible. The search exposes bounded snapshots containing FEN, move, visits, prior, Q value, PV membership, and transposition signature for rendering.

4.10 src/viz

The visualization layer owns raylib resources and UI state:

- **App:** window lifetime, layout, input dispatch, model loading, async evaluation, search preview, MCTS worker, and rendering.
- **BoardView:** board, pieces, coordinates, selection, last move, legal targets, drag piece, and heatmap overlays.
- **Controls:** command button grid with active-state rendering.
- **NnueView, CnnView, Bt4View, ClassicalView:** architecture panels.
- **TreeView:** full-window MCTS graph with mini-board nodes.
- **FenInputDialog, PromotionPicker, EditorPalette, EditorPanel, MoveListView:** focused widgets for the chess workflow.

5. Major Data Members And Ownership

Class	Key data	Ownership / invariant
<code>Position</code>	bitboards, mailbox board, irreversible-state stack, hash history	Owns chess state; bitboards and mailbox are updated together.

Class	Key data	Ownership / invariant
Move	packed <code>uint16_t</code> value	Value type; castling/en-passant meaning is interpreted through <code>Position</code> .
MoveList	fixed array of up to 256 moves	Stack-friendly legal-move container with no heap allocation.
Game	<code>Position</code> , <code>MoveHistory</code>	Single mutation gateway for user-visible play.
MoveHistory	<code>MoveRecord*</code> <code>head/current</code>	Manually managed doubly linked list for undo/redo and move-list rendering.
ActivationSnapshot	map of string keys to tensor entries	Owens copied activation vectors produced by network evaluation.
<code>Tensor<T, Rank></code>	<code>std::vector<T></code> , shape, strides	Generic storage for tensor operations and tests.
<code>lc0_cnn::Network</code>	parsed <code>Weights</code> , policy reverse map	Owens model weights; reverse map is built lazily.
<code>lc0_bt4::Network</code>	<code>Bt4RealWeights</code> , real/synthetic flag	Uses real BT4J weights when loaded; otherwise synthetic fallback.
<code>search::Mcts</code>	node vector, stats vector, signature map	Owens tree state and emits render-friendly snapshots.
App	game, config, sprites, views, futures, threads, caches	Top-level owner of runtime state and raylib window lifetime.
BoardView	non-owning <code>Position*</code> , non-owning <code>PieceSprites*</code>	Renders the position owned by <code>Game</code> or an evaluation override.
TreeView	<code>search::Snapshot</code> , display nodes, parsed mini-boards	Owens display data derived from the latest MCTS snapshot.

6. Runtime Data Flow

Normal board mode:

mouse/keyboard input

- > App
- > Game / Position
- > async architecture evaluation
- > ActivationSnapshot
- > BoardView overlay + active activation view
- > raylib frame

Tree mode:

Start button

- > `App::startMcts`
- > `search::Mcts` worker thread
- > bounded `search::Snapshot` frames
- > TreeView layout
- > selected/followed FEN optional evaluation override

Search preview mode:

Search button / E

- > background alpha-beta style preview
- > selected architecture evaluator when available

- > current preview position
- > activation panel traces that preview position

7. Threading Model

The render loop stays on the main thread. Long-running work is moved off the frame path:

- network evaluation runs through a future and is guarded by a mutex when accessing network objects,
- live search preview runs in an async job with an atomic cancellation flag,
- MCTS tree growth runs in a worker thread and publishes snapshots under a mutex.

The UI copies completed results back into view-owned state before drawing. Search is stopped when the root position changes so stale search output is not applied to a different board.

8. File Formats

Format	Magic	Loader	Purpose
NNUE visualizer model	NNUE	<code>nnue::Loader</code>	Compact HalfKP feature, accumulator, and output weights.
LC0 CNN visualizer model	LC0J	<code>lc0_cnn::Loader</code>	Compact LC0-style CNN weights and policy map.
BT4 real visualizer model	BT4J	<code>lc0_bt4::loadBt4Real</code>	CRTK BT4J v2 transformer weights.
BT4 legacy visual metadata	BT4V	<code>lc0_bt4::Network::load</code>	Synthetic fallback dimension validation.
Config	text key/value	<code>io::Config</code>	Window, assets, models, startup architecture, MCTS, clock.
FEN	text	<code>Fen / FenIo</code>	Position import/export.

All binary model formats are little-endian.

9. UI Layout

The main window is split into:

1. a left board/status area,
2. a right command and activation panel area.

When an activation architecture is selected, the activation view receives most of the lower right panel and the move list becomes a compact footer. When Off is selected, the move list fills the available right panel space. Tree mode replaces the whole window with a toolbar and pan/zoom tree canvas.

10. Error Handling

- Missing NNUE weights fall back to a zero-ish visual path and show status.
- Missing CNN weights disable the real CNN path and use fallback search behavior.
- BT4 loads lazily. If the real BT4J file fails, the network remains usable via the synthetic fallback.
- Invalid FEN input remains in the dialog and displays the parse error.
- Invalid setup-editor positions cannot be applied.
- Search/evaluation exceptions are caught and fall back to Classical evaluation where possible.

11. Testing Strategy

The automated suite covers:

- bitboard helpers, piece/move encoding, legal move generation, perft, make/unmake, hashing, FEN, SAN, draw/status helpers,
- game loop, PGN export, move-history linked list, editor validation,
- config and FEN file I/O,
- tensor ops and activation snapshot storage,
- NNUE, CNN, BT4 loader/forward paths and runtime model smoke tests,
- Classical evaluator terms, WDL, and PST heatmaps,
- MCTS invariants and forced-mate behavior,
- top-level smoke test.

The current local run on 2026-06-08 passes 151 tests with 0 failures.

12. Intentional Limitations

- Human-vs-human play only; no engine opponent mode yet.
- FEN load/save is available; PGN import and file pickers are not implemented.
- Compact CNN and BT4 model files are untrained visualization models, so they demonstrate shape and data flow rather than playing strength.
- Official LC0 protobuf model import is outside the current runtime scope.
- UI tests are documented manually because the project uses an interactive desktop window rather than a headless browser UI.

User Manual

1. Install And Run

Install a C++17 compiler, CMake 3.20 or newer, and the system libraries needed by raylib. On Debian/Ubuntu:

```
sudo apt install build-essential cmake libgl1-mesa-dev libx11-dev \
    libxrandr-dev libxinerama-dev libxcursor-dev libxi-dev
```

Build and launch from the repository root:

```
./scripts/build.sh
./scripts/run.sh
```

The checked-in configuration starts windowed at 1280x800 with the CNN activation panel selected. Edit `config.ini` and restart to change startup size, fullscreen mode, default FEN, board palette, model paths, startup architecture, MCTS settings, fonts, or clock settings.

Before a model demo, make sure the runtime model files exist:

```
./scripts/import_models.sh
```

On Windows, use the packaged build in `dist/cpp-nn-visualizer-windows-x64/` and run either `run-cnnv.bat` or `run-cnnv.ps1`.

2. Basic Chess Play

Move input is mouse based. Click a piece and then click a legal destination, or drag a piece to a legal destination. The board highlights the selected piece, legal targets, and the last move. Promotion opens a picker; choose the promoted piece with the mouse or press **Esc** to cancel the pending selection.

The status area reports the game state. The move list records SAN moves and can be clicked to jump to a previous ply. Undo/redo keeps the linked-list move history intact, so rewinding and replaying does not lose future moves unless a new move is made from the rewind position.

3. Command Buttons

Button	Action
Reset	Restores the configured start position and clears transient UI state.
Flip	Reverses board orientation without changing the position.
Undo	Steps one ply backward through move history.
Redo	Steps one ply forward after an undo.
Random	Plays one random legal move from the current position.
Search	Starts or stops the live search preview; the button uses its active color while search is running.
Load FEN	Opens a text dialog for loading a complete six-field FEN.
Save FEN	Writes the current position to <code>position.fen</code> .
Setup	Opens the board editor.
Edit FEN	Opens the current position as editable FEN text and then enters setup mode.

4. FEN Loading And Saving

Click Load FEN or press **L** to open the FEN dialog. Enter a complete six-field FEN string and press Enter. Invalid input stays in the dialog and shows an inline error.

Click Save FEN or press **S** to write the current position to `position.fen`. There is no file picker yet.

5. Setup Editor

Click Setup or press **T** to enter editor mode. Select a piece from the palette, left-click a square to place it, or right-click a square to erase it. The palette also includes Eraser, Clear, and Startpos controls.

The editor panel controls side to move, castling rights, en-passant target, halfmove clock, and fullmove number. Click Validate before Apply. Apply is enabled only when the edited position is legal. Invalid positions stay in the editor and show the first validation issues.

6. Activation Views

The right-side architecture selector has NNUE, CNN, BT4, Classical, Off, and a Tree tab. Press **A** to cycle NNUE -> CNN -> BT4 -> Classical -> Off -> NNUE. Changing the architecture does not change the chess position.

The activation panel can be zoomed and panned:

Input	Action
Mouse wheel over panel	Zoom toward the cursor.
Right or middle drag	Pan the panel.
+ / -	Zoom about the panel center.
0	Reset panel camera.

NNUE

NNUE shows HalfKP-style active features, raw accumulators, clipped activations, output weights, centipawn value, and learned-weight atlases. Its five internal modes are Overview, Trace, All, Atlas, and Diagram.

CNN

CNN uses the compact LC0J runtime file. It encodes the board into 112 planes, runs the residual trunk, policy head, and WDL/value head, and publishes heatmaps and tensor statistics. Its modes are Overview, Trace, All, Atlas, and Diagram.

BT4

BT4 loads the compact BT4J v2 runtime file when the view is first selected. The default file is a real native transformer with 64 square tokens, 96-dimensional embeddings, 4 encoder blocks, 4 attention heads, policy logits, and a WDL value head. The view derives block and head counts from the loaded tensors, so it is not tied to one hardcoded transformer size.

BT4 modes are Overview, Trace, All, Atlas, and Diagram. The attention-board views show the current pieces on the board so attention movement can be read in chess terms rather than only as matrix cells.

Classical

Classical is model-free. It evaluates the current position with a handcrafted centipawn evaluator and shows the term breakdown, WDL triplet, phase, and piece-square-table heatmaps. The detailed mode shows the heatmaps by piece type.

Off

Off hides the activation panel and leaves the chess board playable.

7. Search Preview

Press Search or E to run a live engine-style preview on the current board. Search stops automatically if the position changes. Esc also stops the search. The selected architecture determines the evaluation route when available:

- CNN uses its real WDL head and policy logits for value/priors.
- BT4 uses its WDL head and fallback priors.
- NNUE uses its centipawn value and fallback priors.
- Classical and Off use the handcrafted evaluator.

8. MCTS Tree Workbench

Press G or click Tree to open the full-window search-tree workbench. Click Start to run PUCT search from the current position. Click Stop to stop it, Fit to frame the tree, and Board or Esc/G to return to the main board.

Tree controls:

Control	Action
visits +/-	Change the playout budget.
cpuct +/-	Change the PUCT exploration constant.
Follow	Trace the currently explored leaf into the activation views.
branches +/-	Limit children shown per node; 0 shows all.
depth +/-	Limit displayed tree depth.

Control	Action
Batch	Collapse sibling leaf groups into summary blobs.
Guides	Toggle per-ply guide lines.
Merge	Merge repeated position signatures for transposition display.

Tree navigation:

Input	Action
Mouse wheel	Zoom.
Left drag	Pan.
Click node	Select that node and trace its FEN into the network views.
Space	Play/pause recorded growth frames.
Left / Right	Step the growth scrubber.
Home / End	Jump to first growth frame / live tree.

9. Keyboard Shortcuts

Shortcut	Action
F	Flip board.
R	Reset to configured start position.
L	Open Load FEN.
S	Save current position to <code>position.fen</code> .
T	Enter setup editor.
A	Cycle activation architecture.
E	Start/stop search preview.
G	Enter/leave tree workbench.
F11	Toggle borderless fullscreen.
Ctrl+Z	Undo one ply.
Ctrl+Y	Redo one ply.
Left / Right	Step backward / forward one ply in board mode; scrub tree frames in tree mode.
Home / End	Jump to start/end of move history in board mode; first/live frame in tree mode.
Esc	Deselect, close modal, stop search, leave editor/tree, or cancel transient input.

10. Configuration

Important `config.ini` keys:

```

window.width = 1280
window.height = 800
window.fullscreen = false
default.fen = rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/RNBQKBNR w KQkq - 0 1
assets.pieces = assets/pieces
board.palette = red
startup.arch = cnn
startup.tree = false

```

```
mcts.visits = 20000
mcts.cpuct = 2.8
model.nnue = models/nnue-halfkp-demo.bin
model.lc0_cnn = models/lc0-cnn-small-112p-4x32-policy4672-wdl3.bin
model.lc0_bt4 = models/lc0-bt4-tiny-96x4x4h.bin
clock.enabled = false
```

startup.arch accepts nnue, cnn, bt4, classical, or off. board.palette accepts red, classic, warm, neural, analysis, heatmap, blue, cool, green, or forest.

11. Restrictions

- Move input is mouse-based; there is no SAN/UCI move-entry box.
- FEN loading expects a complete standard six-field FEN string.
- PGN export exists in the codebase, but PGN import is not implemented.
- There is no engine-opponent play mode yet.
- The bundled CNN and BT4 models are compact untrained visualization models. They demonstrate the real native architecture paths, not upstream engine strength.
- The app reads local compact binary model formats (NNUE, LC0J, BT4J and legacy BT4V fallback meta-data), not official LC0 protobuf files.
- Save FEN writes to `position.fen`; there is no file picker yet.

12. Troubleshooting

- If pieces are missing, check `assets.pieces` and run from the repository root or package directory.
- If model panels report missing weights, check `model.*` paths and rerun `./scripts/import_models.sh`.
- If text input in the FEN dialog is wrong, switch the active keyboard/input method to plain Latin text.
- If the window opens too large or too small, edit `window.width`, `window.height`, or `window.fullscreen` in `config.ini`.